

✓ AG3 - Actividad Guiada 3

Nombre: Irune Urrutia Andres

Link:

[https://colab.research.google.com/drive/1h_VCLtjVhvY33VleYkqZCxpHlaBdgZKj#scrollTo=HtW1iwx2CfaJ\\$0](https://colab.research.google.com/drive/1h_VCLtjVhvY33VleYkqZCxpHlaBdgZKj#scrollTo=HtW1iwx2CfaJ$0)

Github: [https://github.com/iruneurrutia/O3MIAR_ALGORITMOS_ACTIVIDADES\\$0](https://github.com/iruneurrutia/O3MIAR_ALGORITMOS_ACTIVIDADES$0)

```
#Modulo de llamadas http para descargar ficheros
```

```
!pip install requests
```

```
#Libreria del problema TSP: http://elib.zib.de/pub/mp-testdata/tsp/t
```

```
!pip install tsplib95
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.13
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13
Requirement already satisfied: certifi<2017.4.17 in /usr/local/lib/python3.13
Collecting tsplib95
```

```
  Downloading tsplib95-0.7.1-py2.py3-none-any.whl.metadata (6.3 kB)
```

```
Requirement already satisfied: Click>=6.0 in /usr/local/lib/python3.13
```

```
Collecting Deprecated~1.2.9 (from tsplib95)
```

```
  Downloading Deprecated-1.2.18-py2.py3-none-any.whl.metadata (5.7 kB)
```

```
Collecting networkx~2.1 (from tsplib95)
```

```
  Downloading networkx-2.8.8-py3-none-any.whl.metadata (5.1 kB)
```

```
Collecting tabulate~0.8.7 (from tsplib95)
```

```
  Downloading tabulate-0.8.10-py3-none-any.whl.metadata (25 kB)
```

```
Collecting wrapt<2,>=1.10 (from Deprecated~1.2.9->tsplib95)
```

```
  Downloading wrapt-1.17.3-cp313-cp313-manylinux1_x86_64.manylinux_2_28
```

```
  Downloading tsplib95-0.7.1-py2.py3-none-any.whl (25 kB)
```

```
  Downloading Deprecated-1.2.18-py2.py3-none-any.whl (10.0 kB)
```

```
  Downloading networkx-2.8.8-py3-none-any.whl (2.0 MB)
```

```
----- 2.0/2.0 MB 65.0 MB/s eta 0:00
```

```
  Downloading tabulate-0.8.10-py3-none-any.whl (29 kB)
```

```
  Downloading wrapt-1.17.3-cp313-cp313-manylinux1_x86_64.manylinux_2_28
```

```
----- 88.1/88.1 kB 7.2 MB/s eta 0:00
```

```
Installing collected packages: wrapt, tabulate, networkx, Deprecated
```

```
  Attempting uninstall: wrapt
```

```
    Found existing installation: wrapt 2.4.0
```

```
  Uninstalling wrapt-2.4.0:
```

```
    Successfully uninstalled wrapt-2.4.0
```

```
  Attempting uninstall: tabulate
```

```
    Found existing installation: tabulate 0.9.0
```

```
  Uninstalling tabulate-0.9.0:
```

```
    Successfully uninstalled tabulate-0.9.0
```

```
  Attempting uninstall: networkx
```

```
    Found existing installation: networkx 3.6.1
```

```
  Uninstalling networkx-3.6.1:
```

```
    Successfully uninstalled networkx-3.6.1
```

```
ERROR: pip's dependency resolver does not currently take into account
momepy 0.11.0 requires networkx>=3.2, but you have networkx 2.8.8 wh
spopt 0.7.0 requires networkx>=3.2, but you have networkx 2.8.8 whic
bigframes 2.48.0 requires tabulate>=0.9, but you have tabulate 0.8.1
scikit-image 0.25.2 requires networkx>=3.0, but you have networkx 2.8
mapclassify 2.10.0 requires networkx>=3.2, but you have networkx 2.8
Successfully installed Deprecated-1.2.18 networkx-2.8.8 tabulate-0.8
```

```
import tsplib95
import random
from math import e
import urllib.request
```

```
#DATOS DEL PROBLEMA
file = "swiss42.tsp" ;
urllib.request.urlretrieve("https://raw.githubusercontent.com/mastoc
problem = tsplib95.load(file)

#Nodos
Nodos = list(problem.get_nodes())

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
    return problem.get_weight(a,b)

#Devuelve la distancia total de una trayectoria/solucion(lista de n
def distancia_total(solucion, problem):
    distancia_total = 0
    for i in range(len(solucion)-1):
        distancia_total += distancia(solucion[i] ,solucion[i+1] , prob
    return distancia_total + distancia(solucion[len(solucion)-1] ,sol
```

✓ Algoritmo de colonia de hormigas

La función Add_Nodo selecciona al azar un nodo con probabilidad uniforme. Para ser mas eficiente debería seleccionar el próximo nodo siguiendo la probabilidad correspondiente a la ecuación:

$$p_{ij}^k(t) = \frac{[\tau_{ij}(t)]^\alpha [v_{ij}]^\beta}{\sum_{l \in J_i^k} [\tau_{il}(t)]^\alpha [v_{il}]^\beta}, \text{ si } j \in J_i^k$$

$$p_{ij}^k(t) = 0, \text{ si } j \notin J_i^k$$

```

def Add_Nodo(problem, H ,T ) :
    #Mejora:Establecer una funcion de probabilidad para
    # añadir un nuevo nodo dependiendo de los nodos mas cercanos y de
    Nodos = list(problem.get_nodes())
    return random.choice( list(set(range(1,len(Nodos))) - set(H) )

def Incrementa_Feromona(problem, T, H ) :
    #Incrementa segun la calidad de la solución. Añadir una cantidad
    for i in range(len(H)-1):
        T[H[i]][H[i+1]] += 1000/distancia_total(H, problem)
    return T

def Evaporar_Feromonas(T ):
    #Evapora 0.3 el valor de la feromona, sin que baje de 1
    #Mejora:Podemos elegir diferentes funciones de evaporación depend
    T = [[ max(T[i][j] - 0.3 , 1) for i in range(len(Nodos)) ] for j
    return T

```

```

def hormigas(problem, N) :
    #problem = datos del problema
    #N = Número de agentes(hormigas)

    #Nodos
    Nodos = list(problem.get_nodes())
    #Aristas
    Aristas = list(problem.get_edges())

    #Inicializa las aristas con una cantidad inicial de feromonas:1
    #Mejora: inicializar con valores diferentes dependiendo diferentes
    T = [[ 1 for _ in range(len(Nodos)) ] for _ in range(len(Nodos))]

    #Se generan los agentes(hormigas) que serán estructuras de caminos
    Hormiga = [[0] for _ in range(N)]

    #Recorre cada agente construyendo la solución
    for h in range(N) :
        #Para cada agente se construye un camino
        for i in range(len(Nodos)-1) :

            #Elige el siguiente nodo
            Nuevo_Nodo = Add_Nodo(problem, Hormiga[h] ,T )
            Hormiga[h].append(Nuevo_Nodo)

        #Incrementa feromonas en esa arista
        T = Incrementa_Feromona(problem, T, Hormiga[h] )
        #print("Feromonas(1)", T)

    #Evapora Feromonas
    T = Evaporar_Feromonas(T)

```

```
#print("Feromonas(2)", T)

#Seleccionamos el mejor agente
mejor_solucion = []
mejor_distancia = 10e100
for h in range(N) :
    distancia_actual = distancia_total(Hormiga[h], problem)
    if distancia_actual < mejor_distancia:
        mejor_solucion = Hormiga[h]
        mejor_distancia = distancia_actual

print(mejor_solucion)
print(mejor_distancia)

hormigas(problem, 1000)
```

```
[0, 26, 10, 39, 9, 1, 38, 34, 7, 4, 33, 36, 35, 23, 29, 24, 40, 30,
3758
```

